

# TEMA N° 3



## TEMA 3 USO DE IA EN PROGRAMACIÓN Y DESARROLLO



### EN ESTE TEMA SERÁS CAPAZ DE...

1. Integrar herramientas de Inteligencia Artificial (IA) en tu flujo de trabajo diario para acelerar la escritura, comprensión y mejora de código.
2. Diseñar bases de datos complejas y generar datos de prueba adaptados a las necesidades reales de los negocios y organizaciones de nuestra comunidad en Pailón.
3. Refactorizar y optimizar aplicaciones de software garantizando un rendimiento óptimo y manteniendo la ética y la seguridad en cada línea de código escrita.



## PRÁCTICA



¿Te has enfrentado a la presión de tener que entregar un sistema funcional en tiempo récord y no saber por dónde empezar la primera línea de código?

Imagina el siguiente escenario: El CEA Herman Ortiz Camargo, ubicado en nuestro querido Barrio Saó, acaba de estrenar su nuevo y bien equipado módulo educativo. Para celebrar este avance, la dirección ha firmado un convenio con la Asociación de Comerciantes del Barrio Central Fátima y la junta vecinal del Barrio 24 de Septiembre (cerca de la estación de tren). El objetivo es crear una plataforma web que conecte los negocios formales del centro con los vecinos que requieren entregas a domicilio en zonas como el Barrio Residencial Norte, San Antonio y Jardines de Pailón.

Te han asignado a ti, como futuro Técnico en Sistemas Informáticos, la tarea de liderar el desarrollo de esta plataforma. Tienes solo una semana para presentar un prototipo funcional que incluya un registro de usuarios por barrio, un catálogo de productos y un sistema de rutas de entrega. Si intentas escribir cada línea de código desde cero, configurar la base de datos a mano e inventar cientos de usuarios falsos para probar el sistema, el tiempo no te alcanzará. El problema es urgente y los comerciantes de Pailón cuentan con el CEA. ¿Cómo puedes utilizar la Inteligencia Artificial para multiplicar tu velocidad de desarrollo y entregar un software de alta calidad para tu comunidad?



## TEORÍA



El desarrollo de software está viviendo una revolución. Como Técnico en Sistemas Informáticos, la Inteligencia Artificial no viene a reemplazar tu trabajo, sino a potenciarlo. Aquellos que dominen el uso de la IA como un "asistente de programación" (Copilot) serán los profesionales más destacados del mañana. A continuación, desglosaremos cómo la IA interviene en cada fase del desarrollo.



### 3.1. GENERACIÓN DE ESTRUCTURAS DE CÓDIGO REPETITIVAS (BOILERPLATE)

Todo proyecto de software requiere una estructura inicial: archivos de configuración, importaciones de librerías, enrutadores básicos y conexiones al servidor. A esto se le conoce como *Boilerplate*. Escribir esto manualmente en cada nuevo proyecto es una pérdida de tiempo.



La IA puede generar toda la estructura base de nuestra aplicación de entregas para Pailón en segundos. Al pedirle a un modelo de IA (como ChatGPT, Claude o GitHub Copilot) que genere el esqueleto de un servidor en Node.js, nos entrega un código listo para ejecutar.

JavaScript

```
// Importamos el módulo '
express
' para crear nuestro servidor webconst express = require('
express');
// Creamos una instancia de la aplicación expressconst app = express();
// Definimos el puerto donde nuestro servidor escuchará las peticionesconst PORT
= 3000;
// Configuramos la aplicación para que pueda entender datos en formato
JSONapp.use(express.json());
// Creamos la ruta principal o "
raíz" que responde cuando alguien entra al servidorapp.get('/', (req, res) => {
// Enviamos una respuesta de texto simple al cliente res.send('
Bienvenido a la API de Entregas del CEA Herman Ortiz Camargo');
}
);
```



```
// Le indicamos al servidor que comience a escuchar en el puerto
definido app.listen(PORT, () => {
// Imprimimos un mensaje en la consola para saber que el servidor está
funcionando console.log(`Servidor de Pailón ejecutándose en http:
//localhost:{
PORT}
`);
}
);
```

### 3.2. ASISTENCIA EN EL DISEÑO Y CONSULTAS COMPLEJAS DE BASES DE DATOS

Diseñar la arquitectura de una base de datos relacional para que sea eficiente es un reto. En nuestra aplicación, necesitamos conectar a los comerciantes del Barrio Central Fátima con los compradores del Barrio Residencial Norte, Las Misiones y Ovidio Barberi.



Si tenemos dudas sobre cómo estructurar las tablas o cómo hacer consultas (Queries) avanzadas, la IA nos asiste redactando el lenguaje SQL optimizado. Veamos un ejemplo donde le pedimos a la IA crear las tablas para gestionar los barrios de Pailón y los clientes:

SQL

```
-- Creamos la tabla '
barrios
' para almacenar las zonas de PailónCREATE TABLE barrios ( -- ID único y
```





```
auto-incrementable para cada barrio    id_barrio INT AUTO_INCREMENT PRIMARY KEY,
-- Nombre del barrio, no puede quedar vacío    nombre VARCHAR(100) NOT NULL,
-- Distancia aproximada en kilómetros desde el Barrio Saó (CEA)    distancia_km
DECIMAL(5,2));
-- Creamos la tabla '
clientes
' que vivan en esos barriosCREATE TABLE clientes (    -- ID único para el
cliente    id_cliente INT AUTO_INCREMENT PRIMARY KEY,    -- Nombre completo del
cliente    nombre_completo VARCHAR(150) NOT NULL,    -- Teléfono de contacto
telefono VARCHAR(15),    -- Clave foránea que relaciona al cliente con su barrio
id_barrio INT,    -- Establecemos la relación con la tabla '
barrios
'    FOREIGN KEY (id_barrio) REFERENCES barrios(id_barrio));
```

### 3.3. USO DE LA IA PARA EXPLICAR Y COMENTAR BLOQUES DE CÓDIGO COMPLEJOS

Es muy común que como Técnico en Sistemas Informáticos tengas que leer código escrito por otra persona o código antiguo que no tiene documentación. Si un estudiante de una gestión pasada en el CEA dejó un algoritmo confuso para calcular las tarifas de envío desde el Barrio Fátima hasta San Expedito, puedes pegar ese bloque de código en una IA y pedirle: *"Explícame qué hace esta función paso a paso y añade comentarios"*.

La IA analizará la lógica matemática, detectará las variables y te devolverá el mismo código enriquecido con explicaciones, transformando un código incomprensible en un recurso de aprendizaje directo.

### 3.4. REFACTORIZACIÓN ASISTIDA PARA MEJORAR EL RENDIMIENTO DEL CÓDIGO

*Refactorizar* significa cambiar la estructura interna del código sin cambiar su comportamiento externo, con el fin de hacerlo más rápido, más limpio o consuma menos memoria.

Supongamos que escribiste una función para buscar un comercio específico en Pailón utilizando un ciclo que revisa uno por uno todos los registros. Esto funciona si hay 10 comercios, pero si la aplicación crece, el sistema colapsará. La IA puede identificar esta ineficiencia y reescribir el código usando métodos modernos y rápidos.

**Código Original (Lento):**



Python

```
# Definimos una lista de comercios simulando la base de datoscomercios_pailon =
[
    {
        "id": 1, "
        nombre": "
        Tienda María", "
        barrio": "
        Central Fátima"}
]
# Función ineficiente para buscar un comercio
def buscar_comercio_lento(id_buscado):    # Iniciamos un ciclo para recorrer cada
comercio en la lista    for comercio in comercios_pailon:        # Verificamos
si el ID coincide        if comercio["
id"] == id_buscado:            # Si coincide, retornamos el comercio
return comercio    # Si termina el ciclo y no encuentra nada, retorna None
return None
```

**Código Refactorizado por la IA (Optimizado):**

Python

```
# Convertimos la lista en un diccionario (Hash Map) para búsquedas
instantáneascomercios_diccionario = {
    1: {
        "
        nombre": "
        Tienda María", "
        barrio": "
        Central Fátima"}
    }
# Función refactorizada altamente eficiente
def buscar_comercio_rapido(id_buscado):    # Usamos el método .get() que busca la
llave directamente sin recorrer todo    # Si el ID existe lo devuelve, si no,
devuelve None automáticamente    return comercios_diccionario.get(id_buscado)
```

### 3.5. GENERACIÓN AUTOMÁTICA DE DATOS DE PRUEBA (MOCK DATA) PARA TESTEO

Antes de que nuestra aplicación se lance oficialmente en el coliseo municipal del Barrio 27 de Mayo o en los parques infantiles del Barrio Residencial Norte, debemos probarla exhaustivamente. No podemos hacer pruebas con 3 usuarios; necesitamos cientos de registros falsos pero realistas (*Mock data*).



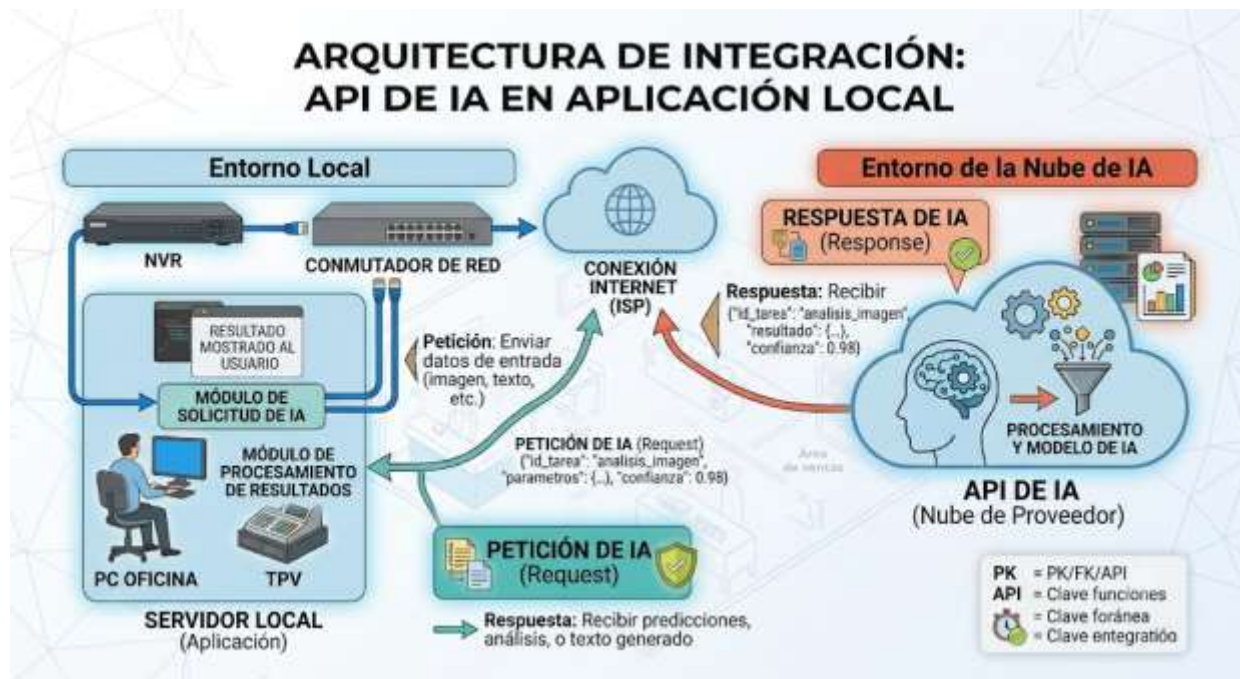
En lugar de inventar nombres durante horas, le damos a la IA un formato JSON o SQL y le pedimos que genere 500 registros con nombres bolivianos, direcciones en los barrios de Pailón (como María Belén, Oto Waver o San José) y números de teléfono ficticios. Esto permite probar cómo reacciona el sistema ante cargas masivas de información.

#### JSON

```
[ {  
  "  
  id": 1,      "  
  nombre": "  
  Carlos Mamani",    "  
  barrio": "  
  Oto Waver",      "  
  telefono": "70001234",    "  
  compras_realizadas": 12  }  
, {  
  "  
  id": 2,      "  
  nombre": "  
  Ana Rodríguez",    "  
  barrio": "  
  San José",      "  
  telefono": "71112233",    "  
  compras_realizadas": 5  }  
]
```

### 3.6. INTEGRACIÓN DE APIS DE INTELIGENCIA ARTIFICIAL EN PROYECTOS LOCALES

Más allá de usar la IA para programar, el verdadero salto de calidad para un Técnico ocurre cuando integra la IA *dentro* del software que está desarrollando. Utilizando APIs (Interfaces de Programación de Aplicaciones) como las de OpenAI, Gemini o Claude, podemos dotar a nuestras aplicaciones de "inteligencia".



Por ejemplo, para el sistema del CEA, podríamos integrar un chatbot impulsado por IA en WhatsApp. Así, un vecino del Barrio Jardines de Pailón podría escribir: *"Necesito que me envíen 2 kilos de pollo desde el mercado central Fátima"*. El sistema, a través de la API, comprendería el lenguaje natural, procesaría la orden automáticamente, calcularía la distancia y le asignaría un repartidor, todo sin intervención humana directa.

### 3.7. SEGURIDAD Y DETECCIÓN DE VULNERABILIDADES EN CÓDIGO GENERADO

Una tendencia crítica actual es la ciberseguridad asistida. Las inteligencias artificiales a veces sufren de "alucinaciones" (inventan funciones o librerías que no existen) o generan código que funciona, pero que es vulnerable a ataques de hackers (como la inyección SQL).

Como Técnico, jamás debes copiar y pegar código generado por IA sin auditarlo. Paradójicamente, podemos usar herramientas de IA especializadas en seguridad (como Snyk o GitHub Advanced Security) para escanear nuestro propio proyecto, detectar puertos abiertos, contraseñas quemadas en el código o fallos en la validación de los datos que ingresan los usuarios. La IA es un gran escritor, pero el Técnico siempre debe ser el revisor experto.





### 3.8. EL FUTURO DEL DESARROLLO: AGENTES DE IA AUTÓNOMOS

La frontera tecnológica actual ya no son solo los "asistentes de código", sino los *Agentes Autónomos* (como Devin AI o AutoGPT). Estas herramientas no solo autocompletan una línea, sino que toman un problema complejo (Ej. "Crea un clon de Facebook para el CEA"), planifican la arquitectura, escriben el frontend, el backend, configuran la base de datos, ejecutan pruebas, corrigen sus propios errores leyendo los registros de la consola, y lo suben a un servidor.

¿Esto significa el fin del Técnico en Sistemas Informáticos? Absolutamente no. Significa que el enfoque de tu carrera en el CEA Herman Ortiz Camargo debe elevarse. Pasarás de ser un simple "picador de código" a un "Arquitecto de Soluciones". Tu trabajo será entender profundamente las necesidades de los vecinos de Pailón, diseñar la lógica empresarial y dirigir a estos agentes de IA para que construyan las herramientas que la comunidad necesita.

### CIERRE TEÓRICO OBLIGATORIO

#### □ Mitos vs. Realidades

| Mito                                                           | Realidad                                                                                                                                        |
|----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>La IA va a reemplazar a los Técnicos en Sistemas.</b>       | <b>Falso.</b> La IA reemplazará a los técnicos que NO usen IA. Los profesionales que dominen estas herramientas multiplicarán su productividad. |
| <b>El código generado por IA siempre es perfecto y seguro.</b> | <b>Falso.</b> La IA puede generar código obsoleto o con fallas de seguridad. El humano siempre debe auditar y probar el resultado final.        |

#### □ Glosario de Términos

1. **Boilerplate:** Código base o estructura repetitiva necesaria para iniciar un proyecto de software sin aportar lógica de negocio.
2. **Refactorización:** Proceso de reestructurar un código fuente, alterando su estructura interna sin cambiar su comportamiento externo.
3. **Mock data:** Datos ficticios generados artificialmente para simular un entorno real y probar el funcionamiento de un sistema.



4. **API (Interfaz de Programación de Aplicaciones):** Conjunto de reglas que permite que dos aplicaciones de software (ej. tu programa y ChatGPT) se comuniquen entre sí.
5. **Alucinación (en IA):** Respuesta generada por un modelo de lenguaje que es falsa o inventada, aunque se presenta con total seguridad.

#### □ **Ponte a Prueba**

1. ¿Cuál es el propósito principal de usar *Mock data* en un proyecto?
  - a) Mejorar el diseño visual de la aplicación.
  - b) Simular grandes volúmenes de usuarios falsos para probar el sistema antes de su lanzamiento.
  - c) Comprimir el código fuente para que ocupe menos espacio.
2. Si un código funciona bien, pero es muy lento y consume mucha memoria, ¿qué técnica asistida por IA deberías aplicar?
  - a) Refactorización.
  - b) Boilerplate.
  - c) Creación de Base de Datos.
3. ¿Qué actitud debe tomar el Técnico al recibir un código generado por IA?
  - a) Copiar, pegar y ejecutarlo directamente en el servidor oficial.
  - b) Auditarlo, entender cada línea y verificar que no existan vulnerabilidades de seguridad.
  - c) Borrarlo y escribirlo todo desde cero manualmente siempre.



### VALORACIÓN



El uso de la Inteligencia Artificial en la programación nos otorga un poder técnico inmenso, pero conlleva una gran responsabilidad social y ambiental, especialmente para nosotros en la población de Pailón.

Primero, hablemos del impacto ambiental. El entrenamiento y uso continuo de modelos de IA requiere centros de datos masivos que consumen millones de litros de agua para refrigeración y cantidades exorbitantes de energía eléctrica. Además, al acelerar el desarrollo de software, los



programas exigen cada vez más recursos de hardware, obligando a instituciones como el CEA Herman Ortiz Camargo y a los vecinos a desechar computadoras antiguas, incrementando la basura electrónica (e-waste). Como Técnicos, debemos comprometernos a escribir código *eficiente*, usando la refactorización para que nuestras aplicaciones funcionen bien incluso en equipos de bajos recursos.

Segundo, la ética y privacidad. Si diseñamos la base de datos de los vecinos del Barrio Jardines de Pailón o de los comerciantes de Central Fátima usando asistencia de IA, debemos ser extremadamente cuidadosos de no alimentar a los modelos de IA públicos (como el ChatGPT gratuito) con datos reales, nombres o números de teléfono de nuestra comunidad. Estos datos podrían quedar almacenados en servidores extranjeros, vulnerando la privacidad de nuestros vecinos. El Técnico responsable protege los datos locales por encima de la conveniencia técnica.



## PRODUCCIÓN



### Reto Final: Sistema de Inventario Deportivo para el Coliseo de Pailón

El Barrio 27 de Mayo, donde se encuentra el coliseo municipal, necesita un sistema urgente para controlar los balones, mallas y uniformes prestados a los jóvenes. Tu misión, como Técnico en Sistemas Informáticos, es iniciar este proyecto utilizando IA.

#### Paso 1: Generación del Boilerplate

Abre tu herramienta de IA preferida (ChatGPT, Gemini, etc.) y utiliza el siguiente *prompt* (instrucción):

*"Actúa como un desarrollador web experto. Genera el código boilerplate en HTML5, CSS y un archivo de JavaScript vacío para una página web llamada 'Inventario Coliseo 27 de Mayo'. Incluye un menú de navegación básico centrado y un diseño moderno."*

Copia el código generado en tu editor de texto (como VS Code) y guarda los archivos.

#### Paso 2: Asistencia en Base de Datos

Pide a la IA que estructure los datos:



*"Escribe el código SQL para crear dos tablas: 'Implementos' (id, nombre, cantidad, estado) y 'Prestamos' (id, nombre\_vecino, barrio\_origen, fecha\_prestamo). Relaciona ambas tablas correctamente."*

Analiza el código generado y guárdalo en un archivo llamado database.sql.

### Paso 3: Generación de Mock Data

Necesitamos probar el sistema sin prestar balones reales aún. Escribe en la IA:

*"Genera un archivo JSON con 10 registros falsos (Mock data) de vecinos del Barrio San Expedito y Barrio San Antonio de Pailón que han tomado prestados balones de fútbol sala en el coliseo."*

Guarda esta información en un archivo datos\_prueba.json.

### Paso 4: Auditoría

Revisa cada línea generada. Añade comentarios en español explicando qué hace cada sección del HTML y del SQL generado. Esta es tu evidencia de que no solo delegaste el trabajo, sino que lo dominas.

## RECURSOS ADICIONALES

1. **MDN Web Docs (Mozilla Developer Network):** La biblioteca más completa y confiable para consultar cualquier etiqueta, función o estilo web generado por una IA y entenderlo a profundidad. <https://developer.mozilla.org/es/>
2. **GitHub Copilot Documentation:** Guía oficial sobre cómo utilizar el asistente de IA más famoso integrado directamente en el editor de código. <https://docs.github.com/es/copilot>
3. **OWASP (Open Worldwide Application Security Project):** Un recurso vital para entender las vulnerabilidades de seguridad que debes buscar cuando auditas código generado por Inteligencia Artificial. <https://owasp.org/>